

Name
Roll No

Mithra Bijumon
24B0040

Page No.:

Date: / /

Assignment -3 Architecting an Intelligent CPU Dispatcher

Part-1 Architecture & Baseline Simulations

Design Question 1

Preemption is necessary to achieve the minimum possible wait time as the scheduler must be able to dynamically switch to shorter jobs whenever they arrive. Non-preemptive SJF makes decisions only when the CPU becomes free, so once a long job starts executing, it cannot be interrupted even if much shorter jobs arrive later.

This creates the convoy effect. For eg. if a large database backup begins running one tick before several tiny web requests arrive, non-preemptive SJF forces all short requests to wait until the backup finishes, causing extremely large waiting times & significantly increasing mean wait time.

In contrast, Shortest Remaining Time First (SRTF) preempts the long-running task as soon as shorter jobs arrive & executes those short jobs first. By preventing short jobs from being blocked behind long ones, SRTF achieves the lowest possible average waiting time.

Part-3 Formulating the Environment

Design Question 2

The new cost function was designed to minimize overall latency while explicitly preventing starvation. It consists of 2 components: a penalty proportional to average waiting time & an additional starvation penalty for

Teacher's Signature:.....

excessively delayed processes.

The 1st term penalizes average wait time, encouraging the agent to reduce overall latency & improve throughput. However, optimizing only this term can lead to pathological behaviour where the agent repeatedly prioritizes short jobs while indefinitely delaying long-running processes.

To prevent this, a starvation penalty was introduced:

$$= \sum_{\text{processes}} (\text{remaining_time} * \max(0, (\text{wait_time} - 50)^2))$$

This penalty activates only after a process has waited more than 50 time units, allowing normal queuing delays without excessive punishment. Beyond this threshold, the penalty grows quadratically with waiting time, making prolonged starvation increasingly expensive. Multiplying by remaining execution time further increases the cost of starving large unfinished jobs.

Additionally, a small inactivity penalty whenever no process could be scheduled, discourages the agent from wasting CPU time through idle actions. A large terminal penalty if the episode exceeded max step limit, forces the agent to complete workloads.

Part-5 Part 5 Mission Control & Visualizing Convergence

Design Question 3

Yes, the agent successfully learned a hybrid scheduling policy. It primarily favored SJF-like decisions to minimize mean wait time & p90 wait time, while occasionally using RR-like behaviours to reduce starvation caused by long-waiting processes. This is reflected in its metrics: the RL agent significantly outperformed both SJF & RR in latency metrics while achieving better fairness than pure SJF. Based on the reward function

the agent performed actions that minimized avg wait, but the quadratic starvation penalty discouraged excessively unfair scheduling.

Code Implementation

The project was implemented using custom RL env simulating CPU scheduling ^{with} dynamically generated workloads of 100 processes. The env state consisted of disarchized queue length, avg remaining burst time, & avg wait time. The action space allowed the agent to choose among the 3 scheduling policies at each step: SRF, FCFS, & Round Robin. A Q-learning agent was trained using epsilon-greedy exploration with reward fn designed to minimize waiting time while penalizing starvation & idle CPU time. Baseline schedulers (FCFS, SRF, RR & Random) were implemented separately for comparative evaluation using mean wait time, p90 wait time & Jain's Fairness Index.

Agent Performance During Training

During training over 200,000 episodes, the agent showed clear convergence, with reward steadily improving & latency metrics decreasing significantly. Mean wait time reduced from 600 to around 110, while p90 wait time reduced from 1500 to 330. Comparative evaluation showed that the RL agent outperformed all baseline schedulers in mean wait time & tail latency, while maintaining better fairness than pure SRF. Thus the agent successfully learned an efficient hybrid scheduling policy.